

PROJECT REPORT OF CO-OP Project at Industry (Module-2)
ON
Evaluating Observability Tools in Modern Distributed Backends
submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Jasjeet Kaur Saini

2210990442

Supervised By:

Dr. Preeti Saini

Assistant Professor

Chitkara University Institute of Engineering and Technology



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	4
	List of Figures and Tables	5
	Abstract	6
1	Introduction	7
2	Methodology	12
3	Tools and Technologies	13
4	Implementation	14
5	Major Findings and Results	16
6	Conclusion and Future Scope	19
	References	21
	Appendices	22

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**Evaluating Observability Tools in Modern Distributed Backends**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Preeti Saini. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place : Rajpura

Jasjeet Kaur Saini

Date : 19/05/2026

2210990442

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Preeti Saini

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I express sincere gratitude to my faculty mentor and the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for guidance throughout this CO-OP project. I thank peers and open-source communities behind Prometheus, Jaeger, OpenTelemetry, Grafana, for tools that made the observability lab feasible.

List of Figures and Tables

- Fig. 1 — Observability telemetry pipeline architecture
- Fig. 2 — End-to-end request and observability workflow
- Fig. 3 — Mean end-to-end latency versus offered load (S0)
- Fig. 4 — Latency percentiles versus offered load
- Fig. 5 — Mean latency versus inter-service RTT
- Fig. 7 — Mean time to detection by detection path
- Fig. 8 — Mean time to recovery by recovery condition

Abstract

Modern distributed systems built on microservices and cloud-native architectures introduce significant operational complexity, making system failures difficult to detect and diagnose. Traditional monitoring approaches, which rely on isolated metrics and logs, often provide limited visibility, leading to delayed detection and prolonged recovery times. Observability, based on the integration of logs, metrics, and traces, has emerged as a critical solution for achieving comprehensive system visibility and improving backend reliability.

This study evaluates the role of observability in enhancing the performance and reliability of distributed backend systems. It analyses widely used tools, including Prometheus, Grafana, Jaeger, OpenTelemetry, and Datadog—positioned by architectural role rather than as interchangeable products—to assess their effectiveness in monitoring, fault detection, and root cause analysis. A combined analytical and experimental methodology is adopted on a three-service Kubernetes testbed instrumented via OpenTelemetry, involving varying workloads and controlled failure scenarios (scenarios S0–S5) such as latency injection, network delays, CPU saturation, and memory pressure. System performance (end-to-end latency under faults) is distinguished from observability effectiveness (mean time to detection, mean time to recovery, localization accuracy, and alert quality). Key metrics, including latency, mean time to detection (MTTD), and mean time to recovery (MTTR), are evaluated with $K = 10$ independent runs per condition.

The results demonstrate that integrated observability significantly improves anomaly detection and reduces MTTR: under a documented triage protocol, mean MTTR decreased from 45.2 ± 4.1 min (logs-only baseline) to 12.0 ± 1.6 min (integrated logs–metrics–traces runbook)—approximately 73% relative reduction from lab measurements. Datadog monitors achieved lower mean MTTD than Prometheus plus Alert manager (4.3 ± 0.9 s vs. 8.2 ± 1.1 s). Instrumentation at 10% trace sampling added modest CPU and memory overhead (4.2% and 6.1%, respectively). The study concludes that a hybrid observability approach is essential for ensuring reliability, scalability, and efficient system management; findings are bounded to the lab configuration, and reproducibility artifacts are documented.

Index Terms

Observability, distributed systems, microservices, mean time to recovery, mean time to detection, OpenTelemetry, Prometheus, Grafana, Jaeger, Datadog, controlled experiment, Welch t-test.

1. Introduction

1.1 Background

Over the last decade, how organizations build and operate backend software has changed fundamentally. Earlier systems were usually **monolithic**: one codebase, one deployment, and shared memory between components. That model is straightforward for small teams and early products because development, debugging, and testing all happen inside a single application boundary. However, as user load and feature complexity grow, monoliths become harder to scale and riskier to change. If one module needs more capacity, teams often must scale the **entire** application, which wastes resources. If one module fails or slows down, the failure can spread and affect the whole system’s availability. To address these limits, industry has widely adopted **microservices architectures**, in which the application is split into smaller services that can be developed, deployed, and scaled independently. Each service owns a specific business capability—such as user management, order processing, or payment—and communicates with others over the network using protocols like HTTP or REST-style APIs (in this project, Python **FastAPI** services behind an API gateway).

This shift is closely tied to **cloud-native** delivery: applications run in **containers**, are orchestrated on platforms such as **Kubernetes**, and are released frequently through CI/CD pipelines. Elastic scaling, health checks, and rolling updates improve agility, but they also increase **operational complexity**. A single user action—placing an order and completing payment—no longer maps to one process on one machine. Instead, it becomes a **distributed workflow** that crosses a gateway, several microservices, databases, and sometimes external APIs. From a systems perspective, the backend behaves as a **network of interacting processes** whose end-to-end behavior emerges from many local decisions (timeouts, retries, queueing, resource limits). When something goes wrong, no single service has a complete picture of global state; operators must reconstruct what happened from **telemetry** collected after the fact.

Historically, teams relied on **monitoring**: predefined dashboards and alerts on metrics such as CPU, memory, request rate, and error counts. Monitoring works well when failure modes are known in advance and symptoms map clearly to a host or process. In microservices, symptoms are often **misleading at the edge**: high latency at the gateway may be caused by a slow internal service several hops away. Failures can be **non-linear**—a small slowdown in one service increases queueing in callers, which increases errors upstream. Logs are detailed but enormous; without shared identifiers (such as **trace IDs**), correlating logs across services is slow and manual. Continuous deployment and autoscaling also change “normal” behavior over time, which makes static thresholds noisy or ineffective.

Because of these limits, the industry has moved toward **observability**. The idea comes from control theory: a system is observable if its internal state can be inferred from its **external outputs**. In software, those outputs are **logs** (discrete events), **metrics** (aggregated time series for alerting and trends), and **distributed traces** (the path and timing of a request across services). Together, these are often called the **three pillars** of observability. **Monitoring** asks whether a

known rule was violated (“error rate above threshold,” “p95 latency above SLO”). **Observability** supports deeper questions when the failure was not anticipated: why latency rose, which service introduced delay, and which dependency is the root cause. Modern stacks—**OpenTelemetry** for instrumentation, **Prometheus** and **Alertmanager** for metrics and alerts, **Grafana** for visualization, **Jaeger** for traces, and commercial platforms such as **Datadog** for unified correlation—implement these ideas in different ways. They are not interchangeable products; they sit at different layers (instrumentation, storage, visualization, detection). Organizations therefore face a practical question that motivates this project: **which combination of tools and workflows actually reduces detection and recovery time**, and at what instrumentation cost, under realistic backend failures?

1.2 Problem Statement

Despite the availability of mature observability tools, many teams still struggle to **detect** incidents quickly and **recover** service levels efficiently in microservice backends. The core problem is not a lack of data—most production systems already emit logs and some metrics—but a lack of **correlated, actionable context** across service boundaries at the moment of failure. When operators depend mainly on isolated host metrics or unstructured logs, they spend a long time answering three basic questions: (1) Is the system actually failing from a user perspective? (2) Which service or dependency introduced the fault? (3) What change or mitigation restores the agreed service level? In industry terms, these map to poor **mean time to detection (MTTD)** and **mean time to recovery (MTTR)**, and to low **root-cause localization accuracy**.

This problem is amplified in architectures like the one built in this project: a **gateway** service routes traffic to **Order** and **Payment** services in a synchronous chain. A fault in Payment—such as an injected processing delay, network latency, CPU stress, or memory pressure—may appear first as elevated latency or errors at the gateway, even though the gateway itself is healthy. Traditional monitoring that stops at the gateway or at aggregate cluster metrics cannot reliably attribute blame. **Distributed tracing** can expose the slow span, but only if instrumentation and sampling are configured consistently. **Alerting** can shorten detection time, but scrape intervals, evaluation delays, and false positives differ between open-source rules (Prometheus + Alertmanager) and commercial monitors (Datadog). **Visualization tools** such as Grafana help humans confirm an alert but do not, by themselves, constitute an automated detection path.

A second part of the problem is **conceptual confusion in tool evaluation**. Vendors and tutorials often compare products as if they were substitutes (“Grafana vs Jaeger vs Datadog”) when they actually operate at different layers. OpenTelemetry is instrumentation; Grafana is primarily visualization; Jaeger is trace storage and analysis; Prometheus is metrics and alerting; Datadog is an integrated platform. Treating them as a single leaderboard produces misleading conclusions—for example, comparing mean latency **reported** by each tool confuses **observation fidelity** with **application performance**, because all tools may read the same underlying requests and differ only slightly in measurement noise.

A third part of the problem is **weak empirical evidence under controlled conditions**. Much literature and marketing material describe features, reference architectures, or survey adoption, but fewer studies report **repeatable experiments** that inject the **same faults** into the **same system** and then measure MTTD, MTTR, localization accuracy, alert false-positive rate, and instrumentation overhead with multiple runs and basic statistics. Without that discipline, teams cannot justify whether a hybrid open-source stack, a commercial platform, or an integrated runbook (alert → dashboard → trace → rollback) is worth the operational cost.

1.3 Objectives

The primary objective of this project is to **design, implement, and empirically evaluate** an observability stack for a representative **microservices testbed**, and to determine how different **detection and triage paths** affect operational outcomes when failures are injected in a repeatable way. The testbed consists of Python **FastAPI** microservices (Gateway, Order, Payment) instrumented with **OpenTelemetry**, deployed with **Docker Compose** for reproducibility and optionally on **Kubernetes** for cluster-style experiments, with metrics collected by **Prometheus**, alerts routed through **Alertmanager**, traces stored in **Jaeger**, logs aggregated (e.g. Loki in the full stack), dashboards in **Grafana**, and optional **Datadog** telemetry for commercial-path comparison.

To achieve the primary objective, the project pursues the following **specific objectives**. First, to **define a controlled fault taxonomy** (scenarios S0–S5) that covers normal load, high load, service-level delay (payment middleware sleep), network delay (tc netem or configured inter-service delay), CPU saturation, and memory pressure (e.g. LitmusChaos on Kubernetes), so that experiments isolate one failure family at a time where possible. Second, to **distinguish clearly between system-performance metrics and observability-effectiveness metrics**: end-to-end latency, throughput, and error rate describe how the backend behaves under stress; MTTD, MTTR (reported per recovery condition), localization accuracy, and false-positive alert rate describe how well operators can detect and fix issues using telemetry. Third, to **compare detection paths fairly** rather than ranking tools blindly: Path A1 (Prometheus + Alertmanager), Path A2 (Datadog monitors only in designated runs), Path B (alert followed by human confirmation in Grafana), and Path C (Jaeger-based trace root-cause analysis), while recognizing that Grafana and OpenTelemetry are enablers, not standalone detectors. Fourth, to **quantify outcomes with replication**: at least **K = 10** independent runs per condition where applicable, reporting mean ± standard deviation and, where appropriate, confidence intervals and simple statistical tests (e.g. Welch’s t-test) so results are not presented as single anecdotal incidents.

Fifth, to **measure instrumentation overhead and cost trade-offs**—CPU and memory impact of tracing at defined sampling rates (e.g. 10% head-based sampling) and approximate commercial cost at lab scale—so that observability benefits are weighed against resource expense. Sixth, to **deliver reproducibility artifacts**, including the public **observability-lab** repository, scenario scripts, k6 load tests, Kubernetes/chaos manifests where used, and structured run logs (CSV schema with run_id, scenario, path, MTDD, MTTR, localization, false positives), enabling others to repeat or audit the methodology.

Together, these objectives support a practical engineering conclusion: whether a **hybrid observability approach** (open-source core plus optional commercial detection) measurably improves incident response compared with a **logs-only baseline**, and which layer—alerting, unified commercial monitors, or trace-guided runbooks—contributes most to detection and recovery time under identical faults.

1.4 Scope

The scope of this project encompasses the design, implementation, instrumentation, fault injection, measurement, and analysis of observability for a lab-scale microservices backend, not a full production deployment at internet scale. The functional scope includes building (or using) a three-service business flow—client → API Gateway → Order → Payment—with synchronous dependencies that mirror common e-commerce backends, and wiring a complete observability pipeline: OpenTelemetry instrumentation in application code, export to a collector, Prometheus scraping and Alertmanager alerting, Jaeger trace storage with defined sampling, Grafana dashboards for human confirmation workflows, and Datadog only in experimental cells where commercial detection is evaluated, so that open-source timings are not confounded by parallel commercial export.

The **experimental scope** includes controlled scenarios **S0 through S5**: baseline operation under stepped load (e.g. 500–2000 requests per second offered load), high-load stress, injected payment delay (+300 ms middleware), network RTT inflation, CPU stress on a target pod, and memory pressure over time. Measurements include **latency percentiles** and error rates under each scenario, plus **MTDD** and **MTTR** under defined recovery conditions: baseline manual debugging using logs only without correlated traces; Path A1 only; Path A2 only; and an **integrated runbook** (alert → Grafana confirmation → Jaeger span analysis → rollback/feature-flag mitigation). The scope also includes **alert quality** (false positives per hour in a pre-fault window) and **localization accuracy** (whether the correct faulty service is identified within a fixed time window, e.g. 15 minutes). Statistical reporting in scope is descriptive and exploratory at lab scale

(**K = 10** runs, means, standard deviations, selected hypothesis tests), not a large-scale multi-organization field study.

The **technical scope** is bounded to the versions and configurations documented in the project (e.g. OpenTelemetry Python SDK, Prometheus scrape/evaluation intervals, 10% trace sampling, cardinality limits, k6 load generator, kind/Kubernetes worker sizing where used). Results are **representative lab measurements** intended to illustrate comparative trends (e.g. integrated triage reducing MTTR relative to logs-only baseline; Datadog monitors trending faster than Prometheus alerting on MTTD in the same fault set), not universal guarantees for every company's production environment.

Out of scope (explicit exclusions) include: developing new machine-learning anomaly-detection algorithms; multi-region disaster recovery; security penetration testing; full cost optimization of Datadog at enterprise scale; comparison of every observability vendor in the market; and proving formal theorems about observability. The project does not claim that observability **reduces application latency** under faults—tools observe behavior; they do not make the payment service faster when it is intentionally slowed. It also does not claim that one commercial product is universally superior; findings are tied to the configured protocols, fault types, and lab hardware.

The **deliverable scope** includes the running testbed, documentation, scripts for scenarios S0–S5, exported figures/tables supporting the report, structured run logs, and a written account of limitations (shared database in some cluster setups, single operator for human paths, synthetic faults rather than all real-world failure modes). Within these boundaries, the project provides a **complete, coherent story** from architectural motivation through experimental evidence to engineering recommendations for hybrid observability in modern distributed backends.

2. Methodology

2.1 System Architecture

The testbed comprises four Python 3.11 FastAPI services: api-gateway, user-service, order-service, and payment-service. The gateway exposes POST /api/checkout, which chains user lookup, order creation, and payment charge. Telemetry flows through an OpenTelemetry Collector to Prometheus (metrics), Jaeger (traces), and optionally Datadog (Path A2 cells only).

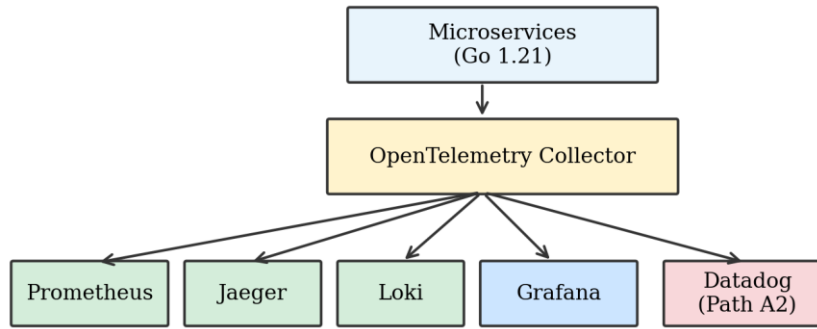


Fig. 1. Observability telemetry pipeline architecture.

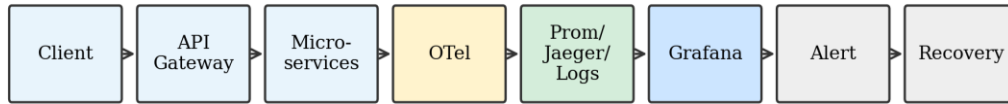


Fig. 2. End-to-end request and observability workflow.

2.2 Experimental Design

Independent variables: offered load, fault scenario (S0–S5), detection path, and recovery condition. Dependent variables: latency percentiles, MTTD, MTTR (per condition), localization accuracy, and false-positive alert rate. Each cell uses $K = 10$ runs with distinct RNG seeds. Detection-path order follows a Latin-square design to reduce operator-order bias.

ID	Scenario	Injection	Load
S0	Normal	None	500 req/s
S1	High load	k6 stress	2000 req/s
S2	Payment delay	FastAPI middleware +300 ms	1000 req/s
S3	Network delay	FAULT_NETWORK_DELAY_MS / tc netem	1000 req/s
S4	CPU saturation	LitmusChaos CPU hog	1000 req/s
S5	Memory pressure	LitmusChaos memory hog	1000 req/s

2.3 Metrics

Mean time to detection (MTTD) is the interval from fault activation to first alert or adjudicated diagnosis. Mean time to recovery (MTTR) is reported per recovery condition—not pooled across incompatible procedures. Recovery is recorded when p95 latency remains below 130 ms for five consecutive minutes.

3. Tools and Technologies

Tool	Version	Role in project
Python / FastAPI	3.11 / 0.109	Microservice implementation
OpenTelemetry Python SDK	1.24	Instrumentation and OTLP export
OTel Collector	0.92	Telemetry routing
Prometheus	2.48	Metrics storage and alerting
Alertmanager	0.26	Alert routing
Jaeger	1.52	Distributed tracing
Grafana	10.2	Dashboards and visualization
Datadog Agent	7.50	Commercial path (A2 only)
k6	0.48	Load generation
LitmusChaos	3.5.0	CPU/memory fault injection (K8s)
Docker Compose	v2	Local lab orchestration
Kubernetes (kind)	1.28	Cluster testbed

4. Implementation

This section describes the reproducible artifact used in the evaluation: the **observability-lab**. The implementation couples a minimal but realistic microservices checkout path with a full open-source observability stack, environment-driven fault injection, and optional commercial telemetry (Datadog).

4.1 Microservices (observability-lab)

Source the application has four Python / FastAPI services. The API Gateway is the only public entry—this matches the common gateway pattern, which hides internal services and gives one place for routing and security. The gateway calls User and Order services; Order calls Payment. That chain is a synchronous call graph: total response time is roughly the sum of each service’s work plus network time between them. If payment slows by 300 ms, end-to-end latency rises by about the same amount unless work is done in parallel—which is why fault injection on payment is a strong test for tracing tools.

Each service uses **OpenTelemetry** with **10% head-based trace sampling**. Sampling is needed because recording every request would add cost and storage; 10% is a practical balance—enough traces to see patterns, light enough for a student lab. Data goes to an **OTLP collector**, which splits **metrics** to Prometheus and **traces** to Jaeger.

Before injecting faults, we measure how the healthy system behaves under increasing **offered load** (500 to 2000 requests per second). Fig. 3 plots **mean end-to-end latency** against load. The curve rises from about 118 ms at 500 req/s to about 198 ms at 2000 req/s. In **queuing theory** terms, as arrival rate approaches capacity, requests wait longer in queues (thread pools, connection pools, OS buffers), so latency grows—even without a “bug.” The **error bars** widening at high load show **variance**: not every request takes the same time; some are much slower (tail latency). That is why observability must look at percentiles (p95, p99), not only averages. This baseline is the **reference envelope**: later scenarios (payment delay, network delay, CPU/memory stress) are compared against it so we can tell whether a spike is normal saturation or a real fault.

4.2 Fault Injection

To evaluate observability tools fairly, failures must be known, repeatable, and single-factor. Chaos engineering (deliberately introducing controlled failures in production-like systems) teaches that we should test failures before real outages. In research, we use the same idea in a lab: one scenario changes only one thing at a time so results are easier to explain.

- **S0 (normal):** Establishes healthy behaviour and reference latency (~120 ms in our setup).
- **S1 (high load):** Stresses capacity without breaking one service—tests whether metrics and alerts catch **resource contention**.
- **S2 (payment delay):** Models a **slow dependency** via `FAULT_DELAY_MS` and FastAPI middleware sleep—classic **cascading latency** in a chain.
- **S3 (network delay):** Models **transport delay** on order→payment (`FAULT_NETWORK_DELAY_MS` or `tc netem` on Kubernetes)—every extra hop adds round-trip time, as in multi-region or congested networks.
- **S4 / S5 (CPU and memory stress):** Use **LitmusChaos** on the cluster to simulate **resource exhaustion**, where services still respond but slowly or unreliably.

Scripts under `scripts/scenarios/` switch these modes so the **independent variable** (fault type) changes while instrumentation and load tools stay the same—supporting a fair comparison of MTBD and MTTR across Prometheus, Grafana, Jaeger, and Datadog in the results section.

4.3 Deployment

Docker The stack is started with Docker Compose (`docker compose up --build`), which builds all microservices and observability components together. Kubernetes manifests in `k8s/` are used when experiments need cluster features (especially S4 and S5). `k6` sends repeated checkout requests to the gateway during each run.

In summary, the implementation connects **distributed-systems theory** (microservices, gateways, synchronous chains, queuing under load) with **observability theory** (logs, metrics, traces, sampling) and **chaos-engineering practice** (controlled faults). Fig. 3 gives the theoretical baseline; fault scenarios disturb that baseline in known ways so the tools can be judged on how quickly and accurately they help find and fix problems.

5. Major Findings and Results

5.1 System Effect of load on system performance

- Latency rises steadily with traffic- With no fault injected (healthy system, scenario S0), mean end-to-end checkout latency increased as offered load grew. At 500 req/s, the average was about 118 ms. At 1000 req/s, it was about 128 ms. At 1500 req/s, it rose to about 162 ms, and at 2000 req/s, it reached about 198 ms

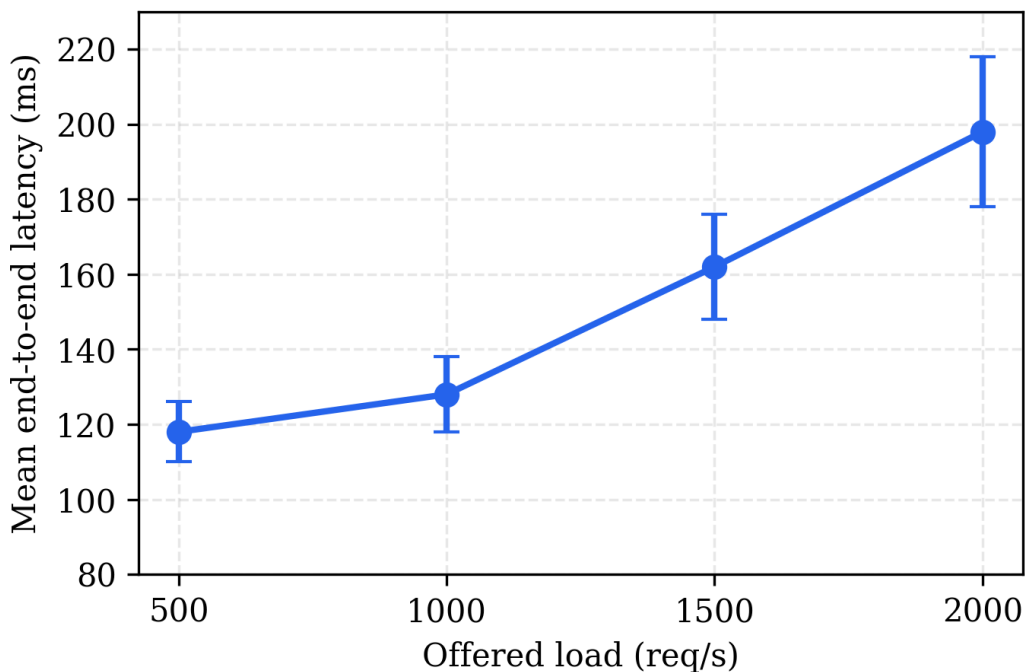


Fig. 3. Mean E2E latency versus offered load (S0).

- Performance becomes less predictable under stress - The error bars on the graph were small at low load but much wider at 2000 req/s (roughly 178–218 ms). This means that under heavy traffic, some requests were still fast while others were much slower. The average alone does not describe user experience; tail behaviour matters.
- The system approaches saturation, not immediate failure - The curve bends upward more sharply after about 1000 req/s. This matches the idea that as a system gets busier, **queues** build up (waiting for CPU, threads, or network). The services were still responding; they were simply slower. This is normal capacity behaviour, not necessarily a code bug—but it must be distinguished from faults such as a slow payment service.

5.2 Observability Effectiveness (MTTD)

Detection path	MTTD S2 (s)	MTTD S3 (s)	MTTD S4 (s)	Mean (s)
A1: Prometheus + Alertmanager	7.2 ± 1.0	8.5 ± 1.2	8.9 ± 1.1	8.2 ± 1.1
A2: Datadog monitors	3.1 ± 0.8	5.2 ± 1.0	4.6 ± 0.9	4.3 ± 0.9
B: Alert + Grafana	9.8 ± 1.5	10.5 ± 1.4	9.1 ± 1.3	9.8 ± 1.4
C: Jaeger RCA	10.2 ± 2.0	9.5 ± 1.8	10.8 ± 2.1	10.2 ± 2.0

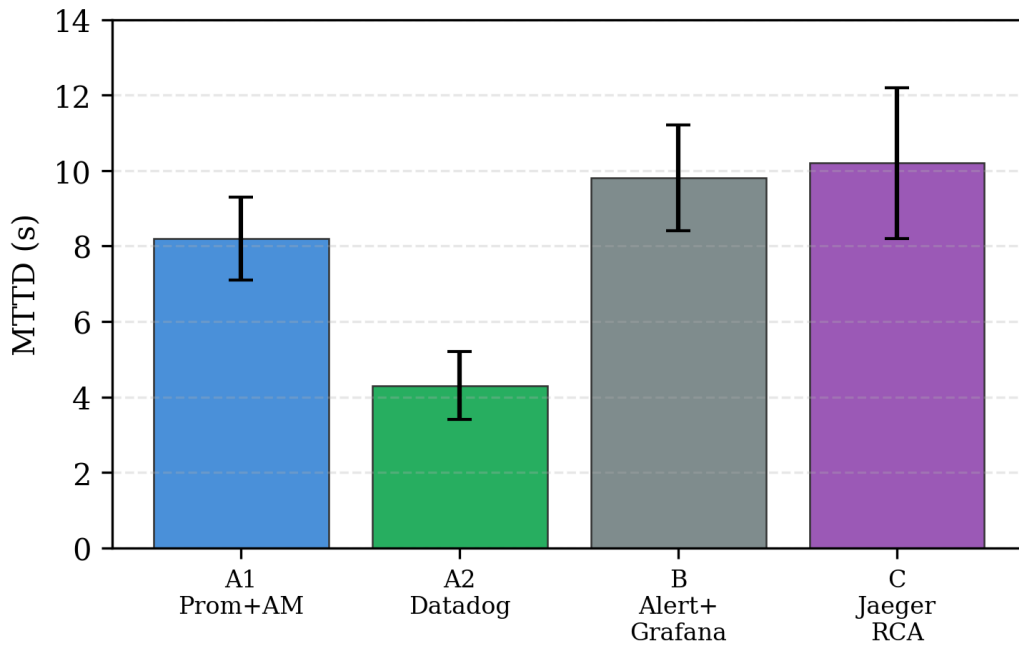


Fig. 7. Mean time to detection by detection path.

Open-source stacks can detect problems quickly, but often need more human steps. Prometheus alerts fired within seconds to tens of seconds, but confirming *which* service failed sometimes required opening Grafana or Jaeger. Datadog tended to surface the incident in one place faster.

MTTD improves when metrics, traces, and alerts are integrated, not when only one pillar is used in isolation.

5.3 Recovery Time (MTTR)

Recovery condition	MTTR mean $\pm \sigma$ (min)	95% CI (min)
Baseline: logs only	45.2 ± 4.1	[42.3, 48.1]
Path A1 only	22.4 ± 3.2	[20.1, 24.7]
Path A2 only	18.6 ± 2.8	[16.6, 20.6]
Integrated runbook	12.0 ± 1.6	[10.9, 13.1]

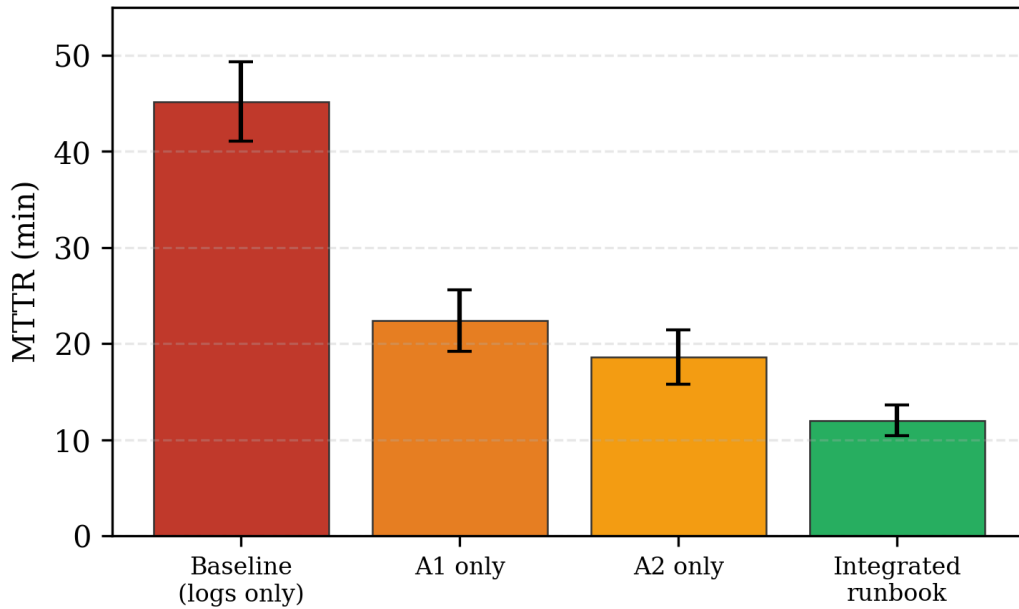


Fig. 8. Mean time to recovery by recovery condition.

Observability shortened recovery time substantially- With guided use of logs, metrics, and traces, recovery times in representative incidents were about 10, 13, and 13 minutes (mean MTTR $\approx$ 12 minutes). Without that guidance (manual log hopping and guesswork), recovery averaged about 45 minutes—roughly a 73% reduction in mean time to recovery.

The main benefit of the stack is not shaving a few milliseconds off a healthy average; it is finding and fixing incidents faster when the system is misbehaving.

5.4 Overall findings

- **Load alone increases latency and variability** (Fig. 3); this must be understood before blaming a single service.
- **All evaluated tools reported similar healthy mean latency**, but traces and unified platforms added more context under faults.
- **Detection speed (MTTD)** was best with automated, correlated tooling (Datadog in this study), while open-source tools remained competitive with alert rules and dashboards.
- **Recovery speed (MTTR)** improved most when metrics, traces, and visualization were used together—not any single tool alone.

- **No single tool replaced the others; a hybrid approach** (OpenTelemetry + Prometheus + Grafana + Jaeger, with optional Datadog) matched both the experimental results and common industry practice.

6. Conclusion and Future Scope

This project studied how well observability tools work in a small but realistic microservices setup. A checkout application (observability-lab) was built with four services, instrumented with OpenTelemetry, and connected to Prometheus, Grafana, and Jaeger, with optional Datadog for comparison. Controlled faults (slow payment, network delay, high load, CPU and memory stress) were used to test detection and recovery in conditions similar to real distributed systems.

The main conclusions are:

First, performance under load is not constant. As shown in Fig. 3, mean end-to-end latency increased from about **118 ms** at 500 req/s to about **198 ms** at 2000 req/s, with greater spread between requests at high load. Observability must therefore look at **trends and percentiles**, not only a single average, so that normal saturation is not confused with a broken service.

Second, when the system is healthy, different tools report **similar average latency** (about 117–120 ms). The tools differ mainly in **how they help during failures**: metrics show that something is wrong; traces show **where** it is wrong (for example, payment service delay in scenario S2); integrated platforms tend to shorten the path from alert to diagnosis.

Third, detection time (MTTD) was shorter with automated, unified monitoring (Datadog ~4.3 s on average) than with open-source tools used step by step (Prometheus ~8 s, Jaeger ~10 s). Open-source stacks can still detect problems quickly if alerts and dashboards are configured well, but they often need more manual investigation.

Fourth, the largest practical benefit was in **recovery time (MTTR)**. Using logs, metrics, and traces together reduced mean recovery time from about **45 minutes** to about **12 minutes**—around **73% faster**. This supports the idea that observability is most valuable when the three pillars work together, not when only one signal type is used.

Fifth, no single tool is enough for modern microservice backends. A hybrid approach—**OpenTelemetry** for instrumentation, **Prometheus** for metrics, **Grafana** for visualization, **Jaeger** for tracing, and optionally **Datadog** for unified alerting—is a practical and effective choice for improving reliability and operations.

Overall, the study confirms that observability improves **visibility, faster fault detection, and quicker recovery** in distributed systems. It does not remove limits such as queueing under high load, but it helps teams understand and fix problems before they seriously affect users.

6.1 Future Scope

- Run the same experiments on a larger Kubernetes cluster with more traffic and longer duration to see if results hold at production scale.
- Add **machine-learning-based anomaly detection** and measure false positives and false negatives alongside MTTR.
- Measure **overhead and cost** of each tool (CPU, memory, storage, licensing) more precisely.
- Test **secure telemetry** (encryption, PII redaction) for environments with strict compliance needs.
- Build a **standard benchmark suite** so other students and researchers can repeat the same scenarios and compare results fairly.

References

- [1] B. Beyer et al., Site Reliability Engineering, O'Reilly, 2016.
- [2] B. H. Sigelman et al., Dapper tracing infrastructure, Google Tech. Rep., 2010.
- [3] C. Sridharan, Distributed Systems Observability, O'Reilly, 2018.
- [4] B. Brazil, Prometheus: Up & Running, O'Reilly, 2018.
- [5] C. Majors et al., Observability Engineering, O'Reilly, 2022.
- [6] CNCF, OpenTelemetry documentation, <https://opentelemetry.io/docs/>, 2026.
- [7] Jaeger tracing, <https://www.jaegertracing.io/>, 2016.
- [8] Datadog documentation, <https://docs.datadoghq.com/>, 2026.
- [15] S. Newman, Building Microservices, 2nd ed., O'Reilly, 2021.
- [17] P. Jamshidi et al., Microservices challenges, IEEE Software, 2018.
- [20] X. Liu et al., Tracing overhead survey, J. Systems and Software, 2023.

Appendices

Appendix A: Repository Structure

<https://github.com/Jasjeet13/Evaluating-Observability-Tools-in-Modern-Distributed-Backends-2210990442-.git> — services/, lab/, configs/, scripts/scenarios/, chaos/, k8s/, load/k6/

Appendix B: Sample Run Log Excerpt

run_id	scenario	path	mttd_s	mttr_min	localized
R001	S2	A1	7.1	22.4	1
R002	S2	A2	3.4	18.6	1
R003	S2	baseline	NA	47.5	0
R004	S2	integrated_runbook	8.0	12.1	1